

2. 管理者・ユーザー管理編

実装順序

要件定義の順番を崩さず、各段階でコードが起動・確認できるように進める。

管理者・ユーザー管理の先行実装

1. 既存のUser、RefreshToken、ログイン、Auth Middlewareを確認する。
2. 既存の `entity/user.go` へ `Suspend` と `Resume` を追加する。
3. AdminAuditLog Entityを実装する。
4. `admin_audit_logs` を作成するGo Migrationを実装する。
5. `users` のUser Entity定義と管理者一覧用Indexが既存Migrationへ含まれていることを確認する。
6. 管理者用Repository Interfaceを定義する。
7. 管理者用Repositoryを実装する。
8. 管理者用Validatorを実装する。
9. 管理者用Usecaseを実装する。
10. `cmd/create_admin/main.go` を実装する。
11. 管理者アカウントを作成する。
12. 管理者アカウントで通常のLogin APIを利用できることを確認する。
13. 管理者権限Middlewareを実装する。
14. 管理者用Controllerを実装する。
15. 管理者用Routerを接続する。
16. `main.go` でRepository、Usecase、Validator、Controller、Middlewareを接続する。
17. ユーザー一覧・詳細APIを確認する。
18. ユーザー利用停止・再開APIを確認する。
19. Frontendの管理者用型とAPI通信を実装する。
20. AdminRouteを実装する。

21. ユーザー一覧画面を実装する。
22. ユーザー詳細画面を実装する。
23. 停止・再開操作を接続する。
24. BackendとFrontendのテストを実行する。
25. ブラウザで管理者・ユーザー管理フローを確認する。

動画編完成後の横断接続

1. ユーザー詳細APIへ投稿動画一覧を接続する。
2. ユーザー利用停止Transactionへ公開動画の `hidden` 化を追加する。
3. `hidden` へ変更した各動画についてAdminAuditLogを作成する。
4. 停止ユーザーの動画処理が完了しても自動公開されないことを確認する。
5. 停止時に `hidden` となった動画がリール、検索、保存一覧へ表示されないことを確認する。
6. ユーザーを再開しても `hidden` 動画が自動公開されないことを確認する。

Video Entityやvideosテーブルを管理者編で仮実装しない。管理者・ユーザー管理機能を先に実装し、動画に依存する処理だけを動画編完成後に接続する。

対象範囲

実装するもの

機能	実装内容
管理者アカウント登録	管理された一度きりの初期化処理でadminを作成する
管理者権限確認	一般ユーザーによる管理者API利用を拒否する
一般ユーザー一覧	名前、メールアドレス、状態、登録日時を表示する
一般ユーザー詳細	ユーザー情報と投稿動画を表示する
ユーザー利用停止	Status変更、TokenVersion増加、Refresh Token失効、監査ログ作成

機能	実装内容
ユーザー利用再開	Status変更、監査ログ作成
管理者画面	一覧、詳細、停止、再開をブラウザから操作する
OpenAPI	管理者APIのRequest、Response、認証、エラーを定義する
テスト	Backend、Frontend、API結合、ブラウザ確認

前提条件

管理者編の実装前に、ユーザー編で次が完成していること。

前提	必要な理由
User Entity	Role、Status、TokenVersionを使用する
RefreshToken Entity	停止時にRefresh Tokenを失効する
usersテーブル	管理者と一般ユーザーを保存する
refresh_tokensテーブル	停止ユーザーのTokenを失効する
ログイン	管理者も通常のログインAPIを使用する
Access Token	管理者APIの認証に使用する
Auth Middleware	DB上のUser状態とTokenVersionを確認する
AuthContext	FrontendでUserとRoleを共有する
ProtectedRoute	未認証状態の画面表示を防ぐ
Request ID	監査ログとAPIログを紐づける
共通エラー形式	管理者APIも同じ形式を使用する

User Entityには `user` と `admin` のRole、`active` と `suspended` のStatus、Access Tokenを一括無効化するTokenVersionが定義。利用停止ではStatusを `suspended` へ変更し、TokenVersionを増加する。利用再開ではStatusのみ `active` へ戻し、TokenVersionは戻さない。

完成状態

管理者編単体で確認する状態

- 管理された方法でadminを作成できる

- 同じ初期化処理を再実行してもadminが増えない
- 一般会員登録APIからadminを作成できない
- adminで通常のLogin APIを利用できる
- 一般ユーザーは管理者APIを利用できない
- 管理者は一般ユーザー一覧を確認できる
- 管理者は一般ユーザー詳細を確認できる
- 管理者は理由付きでユーザーを停止できる
- 停止直後に既存Access Tokenが無効になる
- 停止直後にRefresh Tokenが利用できなくなる
- 停止ユーザーはログインできない
- 管理者は理由付きでユーザーを再開できる
- 再開ユーザーは改めてログインできる
- 停止前のTokenは再利用できない
- 停止・再開操作が監査ログへ保存される

動画編集接続後に確認する状態

- ユーザー詳細に投稿動画が表示される
- 利用停止時に公開動画が **hidden** になる
- 非公開・処理中・削除済み動画は不必要に変更されない
- 変更した各動画について監査ログが作成される
- 停止ユーザーの動画が処理完了後に自動公開されない
- **hidden** 動画がリール、検索、保存一覧に表示されない
- 利用再開しても **hidden** 動画が自動公開されない

全体フロー

1. 管理された初期化処理でadminを作成する。
2. adminが通常のログイン画面からログインする。
3. UserRoleが **admin** のUser情報をAuthContextへ保持する。

4. React Routerが管理者画面への表示可否を判定する。
5. 管理者が一般ユーザー一覧を開く。
6. BackendのAuth MiddlewareがAccess Tokenを検証する。
7. 管理者権限MiddlewareがContext内UserのRoleを確認する。
8. 一般ユーザーだけを一覧取得する。
9. 管理者が対象ユーザーの詳細を開く。
10. ユーザー情報と投稿動画を確認する。
11. 管理者が停止理由を入力する。
12. 対象Userを行ロックする。
13. Userを `suspended` へ変更する。
14. TokenVersionを増加する。
15. Refresh Tokenを失効する。
16. 動画編接続後は公開動画を `hidden` へ変更する。
17. 管理操作を監査ログへ保存する。
18. 全更新が成功した場合だけCommitする。
19. 管理者が再開理由を入力する。
20. 対象Userを行ロックする。
21. Userを `active` へ戻す。
22. 再開操作を監査ログへ保存する。
23. 対象ユーザーが改めてログインする。
24. 新しいAccess TokenとRefresh Token系列を発行する。

Clean Architectureの責務

層	責務	
Entity	Userの状態遷移、AdminAuditLogの不変条件を保持する	
DB	User、Refresh Token、監査ログ、後続のVideo状態を保存する	

層	責務	
Repository	DB取得、行ロック、Transaction、原子的な保存を行う	
Usecase	誰が誰へ何を行うかを判断し、処理全体を組み立てる	
Controller	Path、Query、JSON、HTTP Responseを扱う	
Middleware	認証済みUserとadmin権限を確認する	
Validator	User ID、理由、一覧Queryを検証・正規化する	
Router	URL、HTTPメソッド、Middleware、Controllerを接続する	
Frontend API	管理者APIへの通信をまとめる	
AuthContext	ログインUserとRoleを共有する	
Admin Route	一般ユーザーに管理画面を表示しない	

Transactionの責務

汎用的な `TxManager` は作成しない。

ユーザー利用停止と利用再開では、Usecaseが実行する業務操作を決定し、管理者用Repositoryが行ロックとDB Transactionを担当する。

層	責務
Usecase	誰が誰を停止・再開するかを判断し、実行するRepositoryメソッドを選択する
Entity	UserのRoleとStatusを確認し、正しい状態遷移だけを実行する
Repository	対象Userを行ロックし、複数テーブルの変更を同一Transactionで保存する
Controller	User ID、Reason、Request IDを取得し、Usecaseへ渡す
Middleware	操作Userが認証済みのactiveなadminであることを確認する

ユーザー利用停止のTransaction境界

`AdminUserRepository.SuspendUser` を、利用停止処理における唯一のTransaction境界とする。

同じTransaction内で次を実行する。

1. 対象Userを行ロックする。
2. 対象Userが一般ユーザーかつactiveであることを確認する。

3. User Entityの `Suspend` を実行する。
4. UserのStatus、TokenVersion、UpdatedAtを保存する。
5. 対象Userの未失効Refresh Tokenを失効する。
6. `user_suspend` のAdminAuditLogを作成する。
7. 動画編接続後は対象Userの公開動画を `hidden` へ変更する。
8. 動画編接続後は変更した各動画のAdminAuditLogを作成する。
9. 全処理成功時だけCommitする。
10. 途中で失敗した場合はすべてRollbackする。

利用停止処理では、`RefreshTokenRepository.RevokeByUserID` を別Transactionとして呼び出さない。Refresh Token失効は `AdminUserRepository.SuspendUser` のTransaction内で実行する。

ユーザー利用再開のTransaction境界

`AdminUserRepository.ResumeUser` を、利用再開処理における唯一のTransaction境界とする。

同じTransaction内で次を実行する。

1. 対象Userを行ロックする。
2. 対象Userが一般ユーザーかつsuspendedであることを確認する。
3. User Entityの `Resume` を実行する。
4. UserのStatusとUpdatedAtを保存する。
5. `user_resume` のAdminAuditLogを作成する。
6. 全処理成功時だけCommitする。
7. 途中で失敗した場合はすべてRollbackする。

利用再開では、TokenVersionを戻さない。失効済みRefresh Tokenを復活させない。停止時に `hidden` となった動画も自動公開しない。

構成

既存のClean Architecture構成へ追加する。

```
backend/
├── entity/
│   ├── user.go
│   ├── refresh_token.go
│   ├── admin_audit_log.go
│   └── errors.go
├── db/
│   └── db.go
├── repository/
│   ├── user_repository.go
│   ├── refresh_token_repository.go
│   ├── ratelimit_repository.go
│   └── admin_user_repository.go
├── usecase/
│   ├── user_usecase.go
│   ├── admin_user_usecase.go
│   └── token.go
├── validator/
│   ├── user_validator.go
│   └── admin_user_validator.go
├── controller/
│   ├── user_controller.go
│   └── admin_user_controller.go
├── middleware/
│   ├── auth_middleware.go
│   ├── admin_middleware.go
│   ├── csrf_middleware.go
│   └── ratelimit_middleware.go
├── router/
│   └── router.go
├── migrate/
│   ├── users.go
│   ├── refresh_tokens.go
│   └── admin_audit_logs.go
├── cmd/
│   └── create_admin/
│       └── main.go
└── main.go
```

管理者作成用のエントリーポイントは、通常のHTTPサーバー起動処理と分離する。


```

frontend/
├── src/
│   ├── api/
│   │   ├── client.ts
│   │   ├── user.ts
│   │   └── admin_user.ts
│   ├── auth/
│   │   ├── authContext.ts
│   │   ├── AuthProvider.tsx
│   │   ├── ProtectedRoute.tsx
│   │   ├── AdminRoute.tsx
│   │   ├── tokenStore.ts
│   │   └── useAuth.ts
│   ├── pages/
│   │   ├── SignupPage.tsx
│   │   ├── LoginPage.tsx
│   │   ├── AdminUsersPage.tsx
│   │   ├── AdminUserDetailPage.tsx
│   │   └── NotFoundPage.tsx
│   ├── router/
│   │   └── AppRouter.tsx
│   ├── types/
│   │   ├── user.ts
│   │   └── admin_user.ts
│   └── App.tsx

```

管理者編で変更する既存ファイル

ファイル	変更内容
<code>entity/user.go</code>	<code>Suspend</code> と <code>Resume</code> を追加する
<code>entity/errors.go</code>	管理者権限違反、対象外User、状態競合のDomain Errorを追加する

ファイル	変更内容
<code>repository/user_repository.go</code>	管理者作成で既存の <code>FindByEmail</code> と <code>Create</code> を使用する
<code>router/router.go</code>	管理者用ユーザー管理APIを追加する
<code>main.go</code>	管理者用依存関係を追加する
<code>AuthProvider.tsx</code> <code>authContext.ts</code>	<code>authContext.ts</code> 認証UserのRoleを共有し、AdminRouteが管理画面判定に使用する
<code>AppRouter.tsx</code>	AdminRouteと管理者画面を接続する

Entity

User

管理者・ユーザー管理編では、既存の `entity/user.go` へ利用停止と利用再開の状態遷移を追加する。

Userのフィールド構成は変更しない。

使用する値

項目	値
一般ユーザーRole	<code>user</code>
管理者Role	<code>admin</code>
利用可能Status	<code>active</code>
利用停止Status	<code>suspended</code>

使用する操作

操作	事前条件	状態変更	失敗条件
<code>IsActive() bool</code>	なし	なし	Statusが <code>active</code> の場合だけtrueを返す
<code>IsAdmin() bool</code>	なし	なし	Roleが <code>admin</code> の場合だけtrueを返す
<code>Suspend(now time.Time) error</code>	Roleが <code>user</code> かつ Statusが <code>active</code>	Statusを <code>suspended</code> へ変更	Roleが <code>admin</code> 、または既に

操作	事前条件	状態変更	失敗条件
		し、TokenVersionを1増加し、UpdatedAtへnowを設定する	suspended
Resume(now time.Time) error	Roleが user かつ Statusが suspended	Statusを active へ変更し、 UpdatedAtへnowを設定する	Roleが admin、 または既に active
InvalidateAccessTokens(now time.Time)	Access Tokenの一括無効化が必要	TokenVersionを1増加し、UpdatedAtへnowを設定する	なし

TokenVersionの更新規則

ユーザー利用停止では、Suspend がTokenVersionを1回だけ増加する。

利用停止処理で Suspend を実行した後に、InvalidateAccessTokens を追加で実行しない。

InvalidateAccessTokens は、Refresh Token再利用検知など、利用停止とは別の理由でAccess Tokenを一括無効化する場合に使用する。

守るルール

- 管理者は suspended へ変更しない。
- TokenVersionは減少させない。
- 利用再開時にTokenVersionを停止前の値へ戻さない。
- 停止理由をUserへ保存しない。
- 再開理由をUserへ保存しない。
- 停止理由と再開理由はAdminAuditLogへ保存する。
- Entityはgorm.DB、GORMによるQuery、行ロック、Transaction、HTTP、Cookie、JWTを扱わない。Entityには、DBマッピングに必要な最小限のGORMタグと、JSON出力制御に必要なJSONタグを記載。

AdminAuditLog

管理者が誰に対して、何を、なぜ、いつ行ったかを保存する。作成後に更新・削除しない。

管理者・ユーザー管理編で使用するAction

Action	TargetType	BeforeStatus	AfterStatus
user_suspend	user	active	suspended
user_resume	user	suspended	active
video_hide_by_user_suspension	video	published	hidden

フィールド

フィールド	仕様
ID	DBが発行する監査ログID
AdminUserID	操作した管理者ID
TargetType	user または video
TargetID	対象User IDまたはVideo ID
Action	実行した管理操作
BeforeStatus	操作直前の状態
AfterStatus	操作成功後の状態
Reason	trim後1～500文字
RequestID	APIログと紐づけるRequest ID
CreatedAt	操作成功日時

不変条件

- AdminUserIDは0にしない
- TargetIDは0にしない
- Reasonはtrim後1～500文字
- RequestIDは空文字にしない
- BeforeStatusとAfterStatusを同じ値にしない
- ActionとTargetTypeと状態変更を一致させる
- 作成後に更新しない
- 作成後に削除しない
- Password、Token、Cookie、秘密鍵、署名付きURLを記録しない

AdminAuditLogは状態変更と同じTransaction内で作成する。監査ログ作成に失敗した場合、管理操作自体も失敗させる。

管理者専用Entityは作らない

AdminはUserのRoleであるため、次は作成しない。

- Admin Entity
- Adminテーブル
- AdminPassword
- AdminLogin Entity

UserとAdminを別テーブルへ分離すると、認証処理、Token処理、Role判定が重複する。

DB

使用するテーブル

テーブル	管理者編での用途
<code>users</code>	adminと一般ユーザー、Status、TokenVersion
<code>refresh_tokens</code>	停止ユーザーのRefresh Token失効
<code>admin_audit_logs</code>	停止・再開理由と操作記録
<code>videos</code>	動画編接続後の投稿確認と公開動画非公開

DBではUser、Refresh Token、Video、AdminAuditLogを保存し、User停止時は関連状態を同じTransactionで変更する。管理者監査ログは更新・削除しない。

users

管理者・ユーザー管理編では、既存の `users` テーブルを使用する。

管理者編で必要なUserの保証

項目	保証方法
Roleが <code>user</code> または <code>admin</code>	UserRole定数とUser生成処理
管理者が常にactive	管理者作成UsecaseとUser Entity

項目	保証方法
一般ユーザーだけを停止できる	User Entityの <code>Suspend</code>
activeからsuspendedへの遷移	User Entityの <code>Suspend</code>
suspendedからactiveへの遷移	User Entityの <code>Resume</code>
停止時のTokenVersion増加	User Entityの <code>Suspend</code>
利用再開時にTokenVersionを戻さない	User Entityの <code>Resume</code>
Email重複防止	DBのUNIQUE制約
一覧取得性能	DB Index

Role、Status、TokenVersion、日時整合性のCHECK制約は作成しない。

状態の正しさはEntityとUsecaseで保証し、RepositoryはEntityで確定した状態を保存する。

UNIQUE・INDEX

一般ユーザー一覧では `active` と `suspended` の両方を取得するため、`status` をIndexの並び順へ含めない。

一覧取得では、次の並び順へ固定する。

1. `created_at` の降順
2. `created_at` が同じ場合は `id` の降順

名前	種別	対象	用途
<code>uq_users_email</code>	UNIQUE	<code>email</code>	正規化済みEmailの重複防止
<code>idx_users_admin_list</code>	INDEX	<code>role, created_at DESC, id DESC</code>	管理者用一般ユーザー一覧のCursorページング

更新規則

操作	DB更新
利用停止	<code>status = 'suspended'</code> 、 <code>token_version = token_version + 1</code> 、 <code>updated_at = now</code>
利用再開	<code>status = 'active'</code> 、 <code>updated_at = now</code>
Access Token一括無効化	<code>token_version = token_version + 1</code> 、 <code>updated_at = now</code>

利用停止時のTokenVersion増加は1回だけ行う。

migrate/users.goでは、entity.UserをGORM Migratorへ渡してusersテーブルを作成する。

追加Indexが必要な場合は、GORMのIndexタグまたはMigratorを使用して作成する。

通常のAPI起動時にはMigrationを自動実行しない。

- migrate/users.goでは、entity.UserをGORM Migratorへ渡してusersテーブルを作成する。
- User EntityのGORMタグから、主キー、NOT NULL、EmailのUNIQUE、TokenVersionのDefault 0を反映する。
- 管理者一覧用のidx_users_admin_listもMigrationで作成する。
- Role、Status、TokenVersion、TimestampのCHECK制約は作成しない。

refresh_tokens

利用停止時に対象UserIDの未失効Tokenへ `revoked_at` を設定する。

対象	処理
未失効Token	停止日時を設定する
失効済みToken	変更しない
使用済みToken	必要に応じて失効日時を設定する
期限切れToken	再利用できないため結果に影響させない

平文Refresh Tokenは保存・検索しない。

admin_audit_logs

管理者編の先行実装段階で、 `migrate/admin_audit_logs.go` から作成する。

管理者が一般ユーザーを停止・再開した際に、誰が、誰に対して、何を、なぜ実行したかを記録する。

保存する内容

項目	内容
ID	監査ログを識別するID
AdminUserID	操作した管理者のUser ID

項目	内容
TargetType	操作対象の種類。管理者・ユーザー管理編では <code>user</code> または <code>video</code>
TargetID	操作対象のUser IDまたはVideo ID
Action	実行した管理操作
BeforeStatus	操作前の状態
AfterStatus	操作後の状態
Reason	管理者が入力した停止理由または再開理由
RequestID	APIログと監査ログを紐づけるRequest ID
CreatedAt	管理操作が成功した日時

管理者・ユーザー管理編で使用する操作

Action	対象	変更前	変更後
<code>user_suspend</code>	一般ユーザー	<code>active</code>	<code>suspended</code>
<code>user_resume</code>	一般ユーザー	<code>suspended</code>	<code>active</code>
<code>video_hide_by_user_suspension</code>	停止対象ユーザーの公開動画	<code>published</code>	<code>hidden</code>

保存ルール

- 操作した管理者が特定できること
- 操作対象が特定できること
- Reasonは前後空白を除去した1～500文字とすること
- RequestIDを必須とすること
- 変更前と変更後の状態を同じ値にしないこと
- Action、TargetType、変更前状態、変更後状態の組み合わせを一致させること
- 状態変更と同じTransaction内で作成すること
- 監査ログの作成に失敗した場合は管理操作全体をRollbackすること
- 作成後に更新しないこと
- 作成後に削除しないこと

保存しない情報

- Password
- PasswordHash
- Access Token
- Refresh Token
- TokenHash
- Cookie
- 秘密鍵
- 署名付きURL
- Object Key

videosとの接続

動画編完成後に次を追加する。

- 対象UserIDによる投稿動画取得
- `ready / published / 未削除` 動画の行ロック
- `hidden` への状態変更
- Video単位の監査ログ作成

`private`、`hidden`、処理中、失敗、削除済み動画は停止処理で変更しない。

Repository

管理者用Repository

管理者用Repositoryは、一般ユーザーの管理に必要なDB処理だけを定義する。

メソッド	責務
<code>ListUsers</code>	一般ユーザー一覧を取得する
<code>FindUserDetail</code>	一般ユーザー詳細を取得する
<code>SuspendUser</code>	User停止、Refresh Token失効、監査ログ作成を同一Transactionで行う
<code>ResumeUser</code>	User再開と監査ログ作成を同一Transactionで行う

管理者アカウント作成には、既存の `UserRepository.FindByEmail` と `UserRepository.Create` を使用する。

`AdminUserRepository` へ `CreateAdmin` を重複定義しない。

ListUsers

一般ユーザー一覧はCursor方式で取得する。

Offset方式、ページ番号方式、総件数取得は使用しない。

取得条件

- Roleが `user`
- Statusが `active` または `suspended`
- 管理者を含めない
- PasswordHashを返さない
- Refresh Tokenを結合しない
- `created_at` の降順
- `created_at` が同じ場合は `id` の降順
- Requestから任意の並び順を受け取らない
- 指定件数より1件多く取得し、次ページの有無を判定する

最初のページ

Cursorが指定されていない場合は、次の条件で取得する。

```
role = 'user'  
ORDER BY created_at DESC, id DESC  
LIMIT limit + 1
```

次のページ

Cursorが指定されている場合は、次の条件で取得する。

```
role = 'user'  
AND (  
    created_at < cursor.created_at  
    OR (  
        created_at = cursor.created_at  
        AND id < cursor.id
```

```
)
)
ORDER BY created_at DESC, id DESC
LIMIT limit + 1
```

Cursorの値はGORMのプレースホルダーへ渡す。

Cursorの値をSQL文字列へ直接連結しない。

次ページの判定

`limit = 20` の場合は、DBから最大21件取得する。

DB取得件数	処理
0～20件	取得した全件を返し、 <code>has_more = false</code> とする
21件	先頭20件だけを返し、 <code>has_more = true</code> とする

次ページ判定用の21件目はResponseへ含めない。

`has_more = true` の場合は、Responseへ実際に含める最後のUserを次Cursorの基準とする。

Repositoryが返す情報

項目	内容
Users	Responseへ返す一般ユーザー。最大 <code>limit</code> 件
HasMore	次ページが存在するか
LastCreatedAt	最後に返すUserのCreatedAt
LastID	最後に返すUser ID

RepositoryはBase64形式のCursor文字列を生成しない。

RepositoryはDB取得だけを担当し、Cursor文字列の生成はUsecaseで行う。

FindUserDetail

次を取得する。

User情報

- ID
- Name

- Email
- Status
- CreatedAt
- UpdatedAt

動画編接続後の投稿情報

- Video ID
- Title
- ProcessingStatus
- PublishStatus
- CreatedAt

次を返さない。

- PasswordHash
- TokenVersion
- Refresh Token
- TokenHash
- FamilyID
- Object Key
- 署名付きURL
- Storage認証情報
- Worker内部エラー

SuspendUser

入力

- 操作管理者ID
- 対象User ID
- trim済みReason
- Request ID

- 現在時刻

Transaction

1. Transactionを開始する。
2. 対象Userを行ロックする。
3. 対象Userが存在することを確認する。
4. 対象UserのRoleが `user` であることを確認する。
5. 対象UserのStatusが `active` であることを確認する。
6. User Entityの `Suspend` を実行する。
7. Status、TokenVersion、UpdatedAtを保存する。
8. 対象Userの未失効Refresh TokenへRevokedAtを設定する。
9. `user_suspend` のAdminAuditLogを作成する。
10. 動画編接続後は `ready / published / 未削除` のVideoを行ロックする。
11. 動画編接続後は対象Videoを `hidden` へ変更する。
12. 動画編接続後は各Videoについて `video_hide_by_user_suspension` を作成する。
13. 全処理成功時だけCommitする。
14. 途中で失敗した場合はRollbackする。

利用停止処理では `Suspend` がTokenVersionを増加するため、`InvalidateAccessTokens` を追加で実行しない。

利用停止処理では、`RefreshTokenRepository.RevokeByUserID` を別Transactionとして呼び出さない。

ResumeUser

入力

- 操作管理者ID
- 対象User ID
- trim済みReason
- Request ID
- 現在時刻

Transaction

1. Transactionを開始する。
2. 対象Userを行ロックする。
3. 対象Userが存在することを確認する。
4. 対象UserのRoleが `user` であることを確認する。
5. 対象UserのStatusが `suspended` であることを確認する。
6. User Entityの `Resume` を実行する。
7. StatusとUpdatedAtを保存する。
8. `user_resume` のAdminAuditLogを作成する。
9. 全処理成功時だけCommitする。
10. 途中で失敗した場合はRollbackする。

利用再開では次を行わない。

- TokenVersionを減らさない
- 失効したRefresh Tokenを復活させない
- Access Tokenを発行しない
- hidden動画を自動公開しない
- private動画を自動公開しない

Usecase

管理者用Usecase

操作	内容
CreateAdmin	管理された入力からadminを作成する
ListUsers	一般ユーザー一覧を取得する
GetUserDetail	一般ユーザー詳細を取得する
SuspendUser	一般ユーザーを停止する
ResumeUser	一般ユーザーを再開する

UsecaseはEcho Context、HTTP Status、GORM、Cookie、Transactionの開始・終了を扱わない。

CreateAdmin

管理者アカウントは、公開APIではなく `cmd/create_admin/main.go` から作成する。

処理順

1. `cmd/create_admin/main.go` がName、Email、Passwordを安全な入力元から受け取る。
2. AdminUserValidatorの `ValidateCreateAdmin` を呼び出す。
3. ValidatorがNameをtrimする。
4. ValidatorがEmailをtrim・小文字化する。
5. ValidatorがName、Email、Passwordを検証する。
6. 検証済みの値をAdminUserUsecaseの `CreateAdmin` へ渡す。
7. UsecaseがPasswordをハッシュ化する。
8. UsecaseがRoleを `admin` としてUserを生成する。
9. UsecaseがStatusを `active`、TokenVersionを0へ設定する。
10. UserRepositoryの `FindByEmail` を呼び出す。
11. 同じEmailのadminが存在する場合は重複作成せず成功扱いにする。
12. 同じEmailの一般ユーザーが存在する場合は自動昇格せず競合エラーにする。
13. Userが存在しない場合はUserRepositoryの `Create` を呼び出す。
14. 同時実行によるEmail UNIQUE違反時は既存Userを再取得してRoleを確認する。
15. Password、PasswordHash、Tokenをログや標準出力へ表示しない。

Usecaseが行わないこと

- コマンドライン引数の直接解析
- 標準入力の読み取り
- GORM操作

- TransactionのBegin、Commit、Rollback
- HTTPレスポンス作成
- Cookie操作

ListUsers

1. 操作Userが存在することを確認する。
2. 操作UserのRoleが `admin` であることを確認する。
3. 操作UserのStatusが `active` であることを確認する。
4. Validatorで検証済みの `limit` とCursorを受け取る。
5. Repositoryの `ListUsers` を呼び出す。
6. Repositoryから一般ユーザー一覧と `HasMore` を受け取る。
7. `HasMore` がtrueの場合は、返却する最後のUserの `CreatedAt` と `ID` から `next_cursor` を生成する。
8. `HasMore` がfalseの場合は、`next_cursor` を設定しない。
9. 一般ユーザー一覧、`next_cursor`、`has_more` をControllerへ返す。

取得結果が0件の場合はCursorを生成しない。

Cursorの生成

Cursorには次の値を格納する。

項目	内容
<code>created_at</code>	最後に返したUserのCreatedAt
<code>id</code>	最後に返したUser ID

Cursor内部の例：

```
{
  "created_at": "2026-07-16T10:30:45.123456Z",
  "id": 125
}
```

`created_at` はRFC3339Nano形式とする。

Cursor内部データをJSONへ変換し、Base64のRaw URL Encodingで文字列化する。

Paddingは使用しない。

Cursorは認証情報や認可情報として使用しない。

GetUserDetail

1. 操作Userがactiveなadminであることを確認する。
2. 対象User IDをRepositoryへ渡す。
3. 対象Userが存在することを確認する。
4. 対象Roleが `user` であることを確認する。
5. User詳細を返す。
6. 管理者Userを管理対象として返さない。

SuspendUser

1. 操作Userが存在することを確認する。
2. 操作Userがactiveなadminであることを確認する。
3. 操作User自身を対象にしないことを確認する。
4. ReasonとRequest IDが設定されていることを確認する。
5. RepositoryのSuspendUserを呼ぶ。
6. 更新後UserをControllerへ返す。

対象Userの最終的なRole・Status確認は、競合を防ぐためRepositoryが行ロック後に再確認する。

ResumeUser

1. 操作Userが存在することを確認する。
2. 操作Userがactiveなadminであることを確認する。
3. 操作User自身を対象にしないことを確認する。
4. ReasonとRequest IDが設定されていることを確認する。
5. RepositoryのResumeUserを呼ぶ。

6. 更新後UserをControllerへ返す。

Controller

管理者用Controller

操作	HTTP入力	Usecase
ListUsers	Query	ListUsers
GetUserDetail	Path User ID	GetUserDetail
SuspendUser	Path User ID、JSON Reason	SuspendUser
ResumeUser	Path User ID、JSON Reason	ResumeUser

管理者作成用Controllerは作らない。

ListUsers

1. Auth MiddlewareがContextへ保存した操作Userを取得する。
2. Query Parameterの `limit` を取得する。
3. Query Parameterの `cursor` を取得する。
4. Validatorの `ValidateUserListQuery` を呼び出す。
5. 検証済みの一覧条件をUsecaseへ渡す。
6. Usecaseから一般ユーザー一覧、`next_cursor`、`has_more` を受け取る。
7. 一覧Responseへ変換する。
8. `200 OK` を返す。

Controllerは次を行わない。

- CursorをDecodeしない
- Cursorを生成しない
- SQL条件を組み立てない
- 並び順を決定しない
- `limit + 1` 件の判定を行わない

GetUserDetail

1. Contextから操作Userを取得する。
2. Pathから対象User IDを取得する。
3. ValidatorでUser IDを検証する。
4. Usecaseを呼ぶ。
5. User詳細Responseへ変換する。
6. **200 OK** を返す。

SuspendUser

1. Contextから操作Userを取得する。
2. ContextからRequest IDを取得する。
3. Pathから対象User IDを取得する。
4. JSONからReasonだけを受け取る。
5. User IDを検証する。
6. Reasonを検証・trimする。
7. Usecaseを呼ぶ。
8. 更新後のID、Status、UpdatedAtを返す。
9. **200 OK** を返す。

Role、Status、TokenVersionをRequest Bodyから受け取らない。

ResumeUser

停止と同じ境界処理を行い、UsecaseのResumeUserを呼ぶ。

Middleware

Auth Middleware

ユーザー編で実装したAuth Middlewareを使用する。

確認するもの。

- Authorization Header

- Bearer形式
- JWT署名
- 有効期限
- JWT内User ID
- DB上のUser
- User Status
- JWT内TokenVersionとDB内TokenVersion

正常なUserをEcho Contextへ保存する。

管理者権限Middleware

Auth Middlewareの後に実行する。

1. ContextからUserを取得する。
2. Userが存在しなければ401を返す。
3. Auth Middlewareがactive状態を確認済みであることを前提にする。
4. UserのRoleがadminか確認する。
5. adminの場合だけControllerへ進む。
6. userの場合は403を返す。
7. 権限違反を構造化ログへ記録する。
8. Access TokenやCookieをログへ記録しない。

JWT内のRole Claimだけを使用せず、Auth MiddlewareがDBから取得した現在のUserを使用する。

Body Limit

停止・再開のJSON Requestへ共通の小容量Body Limitを適用する。

CSRF

現在の認証方式では、管理者APIはAuthorization HeaderのAccess Tokenを使用するため、Refresh Token Cookieを送信しない。したがって、管理者APIへCSRF Middlewareは適用しない。

将来Access TokenをCookie認証へ変更した場合は、管理者の状態変更APIにもCSRF対策を追加する。

Validator

管理者用Validatorは、管理者作成入力、User ID、Reason、一覧Queryを検証・正規化する。

メソッド	検証対象
<code>ValidateCreateAdmin</code>	管理者名、Email、Password
<code>ValidateUserID</code>	PathのUser ID
<code>ValidateReason</code>	停止理由、再開理由
<code>ValidateUserListQuery</code>	一覧取得条件

ValidatorはDBへアクセスしない。

管理者作成入力

項目	条件
Name	trim後1～50文字
Email	trim・小文字化後、1～254文字、有効なEmail形式
Password	ユーザー認証編と同じ条件
Role	入力として受け取らない
Status	入力として受け取らない

User ID

次を拒否する。

- 空文字
- 数字以外
- 0
- 負数
- `math.MaxInt64` を超える値

Reason

次を行う。

- 前後空白を除去する
- trim後1～500文字を許可する
- 空文字を拒否する
- 空白だけを拒否する
- 501文字以上を拒否する

Reason内のHTML文字列は文字列として扱う。FrontendでHTMLとして実行しない。

一覧Query

一般ユーザー一覧はCursor方式へ固定する。

Query Parameter

項目	必須	型	条件
<code>limit</code>	任意	integer	1～100。未指定時は20
<code>cursor</code>	任意	string	前回Responseの <code>next_cursor</code> 。最初の取得では指定しない

次のQuery Parameterは使用しない。

- `offset`
- `page`
- `sort`
- `order`

Requestから任意のORDER BY文字列を受け取らない。

limitの検証

入力	結果
未指定	20を使用する
1～100	許可する
0	400 Bad Request
負数	400 Bad Request

入力	結果
101以上	400 Bad Request
数字以外	400 Bad Request

Cursorの構造

Cursorは専用構造体へDecodeする。

項目	型	条件
<code>created_at</code>	<code>time.Time</code>	有効な日時
<code>id</code>	<code>uint64</code>	1以上、 <code>math.MaxInt64</code> 以下

CursorのDecodeに `map[string]interface{}` や `any` を使用しない。

Cursorの検証

1. Cursorが空の場合は最初のページとして扱う。
2. Base64 Raw URL EncodingとしてDecodeする。
3. JSONを専用Cursor構造体へDecodeする。
4. JSONの不明フィールドを拒否する。
5. `created_at` が有効な日時か確認する。
6. `id` が1以上か確認する。
7. `id` が `math.MaxInt64` 以下か確認する。
8. 不正なCursorは `400 Bad Request` として拒否する。

ValidatorはCursorに含まれるUserがDBに存在するか確認しない。

ValidatorはDBへアクセスしない。

Router

API契約

HTTP	URL	内容
<code>GET</code>	<code>/admin/users</code>	一般ユーザー一覧
<code>GET</code>	<code>/admin/users/:user_id</code>	一般ユーザー詳細

HTTP	URL	内容
PATCH	/admin/users/:user_id/suspend	一般ユーザー利用停止
PATCH	/admin/users/:user_id/resume	一般ユーザー利用再開

管理者アカウント作成用APIは登録しない。

Middleware接続

API	Route別Middleware
GET /admin/users	Auth、Admin
GET /admin/users/:user_id	Auth、Admin
PATCH /admin/users/:user_id/suspend	Auth、Admin
PATCH /admin/users/:user_id/resume	Auth、Admin

共通Middlewareは既存のユーザー認証編と同じものを使用する。

1. Request ID
2. Recover
3. Logger
4. Security Header
5. CORS
6. Body Limit
7. Route別Middleware
8. Controller

要件定義では、管理者による停止・再開APIはRate Limit対象として定義されていないため、本仕様書では管理者APIへ新しいRate Limitを追加しない。

将来Rate Limitを追加する場合は、先に要件定義へ対象API、識別単位、制限値、補充速度を追記してから実装する。

管理者アカウント登録

採用方法

環境	方法
本番・検証環境	専用の一度きり初期化コマンド
開発・テスト	管理されたSeedまたは初期化コマンド
通常サーバー起動	実行しない

本番起動時にSeedを自動実行しない。

一般会員登録APIからadminを作成しない。

管理者登録用のHTTP APIとFrontend画面は作成しない。

処理フロー

1. `cmd/create_admin/main.go` を実行する。
2. Name、Email、Passwordを安全な方法で取得する。
3. Validatorが入力値を検証・正規化する。
4. UsecaseがPasswordをハッシュ化する。
5. Roleを `admin` としてUserを生成する。
6. Statusを `active` へ設定する。
7. TokenVersionを0へ設定する。
8. 正規化済みEmailで既存Userを確認する。
9. 同じEmailのadminが存在する場合は新規作成しない。
10. 同じEmailの一般ユーザーが存在する場合は処理を拒否する。
11. Userが存在しない場合だけ保存する。
12. 通常のLogin APIでログインできることを確認する。
13. 管理者APIと管理者画面を利用できることを確認する。

セキュリティ

- Passwordをソースコードへ記載しない。
- PasswordをShell履歴へ不用意に残さない。
- Passwordを標準出力へ表示しない。
- PasswordHashを標準出力へ表示しない。
- 管理者認証情報をGitへコミットしない。

- 既存の一般ユーザーを自動的にadminへ昇格しない。
- 初期化再実行時に既存adminのPasswordHashを自動更新しない。

API Response

ユーザー一覧

Request

最初のページを取得する。

```
GET /admin/users?limit=20
```

次のページを取得する。

```
GET /admin/users?limit=20&cursor={next_cursor}
```

Frontendは `next_cursor` の内容を解釈・変更せず、そのまま次のRequestへ使用する。

Response

```
{
  "items": [
    {
      "id": 125,
      "name": "山田 太郎",
      "email": "taro@example.com",
      "status": "active",
      "created_at": "2026-07-16T10:30:45Z"
    }
  ],
  "next_cursor": "eyJjcmVhdGVkX2F0IjoimjAyNi0wNy0xNlQxMDozMDo0NS4xMjM0NTZaIiwiaWQiOi0jEjN",
  "has_more": true
}
```

Response項目

項目	型	仕様
<code>items</code>	array	一般ユーザー一覧。0件でも <code>null</code> ではなく空配列
<code>next_cursor</code>	stringまたはnull	次ページがある場合だけ設定する
<code>has_more</code>	boolean	次ページが存在する場合はtrue

itemsの各項目

項目	内容
<code>id</code>	User ID
<code>name</code>	ユーザー名
<code>email</code>	メールアドレス
<code>status</code>	<code>active</code> または <code>suspended</code>
<code>created_at</code>	登録日時

次ページが存在しない場合は次の状態とする。

```
{
  "items": [],
  "next_cursor": null,
  "has_more": false
}
```

最終ページに一般ユーザーが存在する場合は、そのUserを `items` へ含めたうえで、`next_cursor = null`、`has_more = false` とする。

`total` は返さない。

一覧取得時に `COUNT(*)` は実行しない。

返さないもの

- Role
- PasswordHash
- TokenVersion
- Refresh Token
- TokenHash

一覧対象が一般ユーザーだけであるため、RoleをFrontendへ返さない。

ユーザー詳細

返すもの。

- ID
- Name
- Email
- Status
- CreatedAt
- Videos

Videosの各項目。

- ID
- Title
- ProcessingStatus
- PublishStatus
- CreatedAt

動画編実装前は `videos: []` とする。

停止・再開

返すもの。

- ID
- Status
- UpdatedAt

TokenVersionはFrontendへ公開しない。

エラー

HTTP	条件
400 Bad Request	User ID、Reason、一覧Queryが不正
401 Unauthorized	未認証、Token期限切れ、署名不正、TokenVersion不一致

HTTP	条件
403 Forbidden	一般ユーザーによる管理API利用、管理者を停止・再開しようとした
404 Not Found	対象一般ユーザーが存在しない
409 Conflict	既に停止中、既にactiveなど現在状態が操作条件と一致しない
413 Content Too Large	共通Body Limitを超過した
500 Internal Server Error	DB、Transaction、監査ログ保存等の内部エラー

APIレスポンスへ次を含めない。

- SQL
- DB制約名
- Stack Trace
- Password
- PasswordHash
- Access Token
- Refresh Token
- TokenHash
- Cookie
- Object Key
- 秘密鍵

Frontend

実装順

1. 管理者用Response型を追加する。
2. 管理者API通信を追加する。
3. AdminRouteを追加する。
4. ユーザー一覧画面を作成する。
5. ユーザー詳細画面を作成する。

6. 停止処理を接続する。
7. 再開処理を接続する。
8. AppRouterへ接続する。
9. Frontendテストを実行する。
10. ブラウザ確認を行う。

Frontend URL

URL	画面
<code>/admin/users</code>	一般ユーザー一覧
<code>/admin/users/:user_id</code>	一般ユーザー詳細

AdminRoute

状態	処理
認証確認中	Loading
未認証	Login画面へ遷移
Roleがuser	管理画面を表示しない
Roleがadmin	管理画面を表示する

FrontendのRole判定は画面表示制御であり、BackendのAdmin Middlewareを代替しない。

ユーザー一覧画面

表示するもの

- Name
- Email
- Status
- CreatedAt
- 詳細画面への導線

adminは一覧へ表示しない。

保持する状態

状態	内容
<code>items</code>	取得済みの一般ユーザー一覧
<code>nextCursor</code>	次ページ取得用Cursor
<code>hasMore</code>	次ページが存在するか
<code>isLoading</code>	API通信中か
<code>error</code>	APIエラー

最初の取得

1. `items` を空配列へ初期化する。
2. `nextCursor` を `null` へ初期化する。
3. `hasMore` を `false` へ初期化する。
4. `GET /admin/users?limit=20` を呼び出す。
5. Responseの `items` を表示する。
6. Responseの `next_cursor` を `nextCursor` へ保存する。
7. Responseの `has_more` を `hasMore` へ保存する。

次ページの取得

1. `hasMore` が `true` であることを確認する。
2. `nextCursor` が存在することを確認する。
3. `isLoading` が `false` であることを確認する。
4. `GET /admin/users?limit=20&cursor={nextCursor}` を呼び出す。
5. 取得した `items` を既存一覧の末尾へ追加する。
6. 新しい `next_cursor` を保存する。
7. 新しい `has_more` を保存する。

二重取得防止

- `isLoading = true` の場合は追加Requestを送信しない
- 同じCursorを複数回送信しない

- Componentの再描画だけを理由に次ページを再取得しない
- `hasMore = false` の場合は追加取得しない
- 不正Cursorが返った場合にFrontendでCursorを加工しない
- 一覧再読込時は `items` とCursorを初期化する

ユーザー詳細画面

表示する。

- Name
- Email
- Status
- CreatedAt
- 投稿動画
- Reason入力欄
- 停止または再開ボタン

Status	表示操作
active	利用停止
suspended	利用再開

操作時

- Reason未入力では送信しない
- 送信中はボタンを無効化する
- 二重送信を防止する
- 成功後はAPI結果でStatusを更新する
- 停止成功後は詳細を再取得する
- 再開成功時にVideo状態をpublishedへ変更しない
- 409時は「既に状態が変更されています」と表示して再取得する

XSS

Reactの通常の文字列出力を使用する。

次を行わない。

- `dangerouslySetInnerHTML`
- NameをHTMLとして挿入する
- Video TitleをHTMLとして挿入する
- ReasonをHTMLとして表示する
- API ErrorをHTMLとして表示する

OpenAPI

次を定義する。

- `GET /admin/users`
- `GET /admin/users/{user_id}`
- `PATCH /admin/users/{user_id}/suspend`
- `PATCH /admin/users/{user_id}/resume`

GET /admin/usersのQuery Parameter

limit

項目	内容
name	<code>limit</code>
in	query
required	false
type	integer
minimum	1
maximum	100
default	20
description	1回に取得する一般ユーザー件数。省略時は20、最大100。

cursor

項目	内容
name	<code>cursor</code>

項目	内容
in	query
required	false
type	string
description	前回Responseで返された次ページ取得用Cursor。内容を変更せず、そのまま指定する。

GET /admin/usersのResponse

項目	必須	内容
<code>items</code>	必須	一般ユーザー一覧
<code>next_cursor</code>	必須	次ページが存在する場合はCursor、最終ページではnull
<code>has_more</code>	必須	次ページが存在する場合はtrue

description

一般ユーザーを登録日時の新しい順で取得する。取得順はCreatedAt降順、ID降順で固定する。次ページが存在する場合はnext_cursorを返す。FrontendはCursorの内容を変更せず、次のRequestへ使用する。

Error Response

- `400 Bad Request`
- `401 Unauthorized`
- `403 Forbidden`
- `500 Internal Server Error`

`400 Bad Request` には次を含める。

- limitが範囲外
- limitが数字ではない
- CursorのBase64 Decode失敗
- CursorのJSON Decode失敗
- Cursorの日時が不正

- CursorのIDが不正

Decode失敗の内部詳細やCursor内部構造をエラーResponseへ含めない。

テスト

管理者作成

- adminを作成できる
- Roleがadmin
- Statusがactive
- TokenVersionが0
- PasswordがHash化される
- Passwordが平文保存されない
- 再実行で重複しない
- 既存adminを変更しない
- 同じEmailのuserを昇格しない
- 同時実行でも重複しない

Admin Middleware

- active adminは通過する
- active userは403
- Tokenなしは401
- 期限切れTokenは401
- TokenVersion不一致は401
- JWT Role改ざんでは通過しない
- 秘密情報をログへ出さない

ユーザー一覧

Validator

- limit未指定時に20になる
- limitが1で成功する
- limitが100で成功する
- limitが0で失敗する
- limitが負数で失敗する
- limitが101で失敗する
- limitが数字以外で失敗する
- Cursor未指定で最初のページとして扱う
- 正しいCursorをDecodeできる
- Base64形式が不正なCursorを拒否する
- JSON形式が不正なCursorを拒否する
- 不明フィールドを持つCursorを拒否する
- created_atが不正なCursorを拒否する
- IDが0のCursorを拒否する
- IDが `math.MaxInt64` を超えるCursorを拒否する

Repository

- 一般ユーザーだけを取得する
- adminを含めない
- activeとsuspendedを含める
- CreatedAt降順、ID降順で取得する
- 最初のページを取得できる
- Cursorより後のUserだけを取得する
- CreatedAtが同じ場合はIDの小さいUserを次ページへ取得する
- `limit + 1` 件取得して次ページを判定する
- 判定用の追加1件をResponse対象へ含めない
- ページ境界で同じUserを重複取得しない
- ページ境界でUserを取得漏れしない

- Cursorの値をSQL文字列へ直接連結しない
- PasswordHashを返さない
- Refresh Tokenを結合しない

Usecase

- `HasMore = true` の場合にNextCursorを生成する
- `HasMore = false` の場合にNextCursorを生成しない
- 取得結果0件の場合にCursorを生成しない
- Responseへ返す最後のUserからCursorを生成する
- CursorのCreatedAtをそのまま使用する。
- CursorをBase64 Raw URL Encodingで生成する

Controller

- limit未指定で正常取得できる
- cursor未指定で正常取得できる
- 不正limitで400を返す
- 不正Cursorで400を返す
- 成功時にitems、next_cursor、has_moreを返す
- 0件の場合もitemsを空配列で返す

Frontend

- 最初のページを取得できる
- next_cursorを保存できる
- 次ページのitemsを既存一覧へ追加できる
- has_moreがfalseの場合は追加取得しない
- isLoading中は二重Requestを送信しない
- 同じCursorを重複送信しない
- 一覧再読込時にitemsとCursorを初期化する

ユーザー詳細

- 一般ユーザーを取得できる
- 存在しないUserは404
- adminは管理対象外
- 動画編前はVideosが空配列
- 動画編後は投稿動画を取得できる
- 削除済み動画を通常表示しない
- Object Keyを返さない

利用停止

- active userを停止できる
- Statusがsuspendedになる
- TokenVersionが1増加する
- Refresh Tokenが失効する
- User監査ログが作成される
- 管理者を停止できない
- 既にsuspendedの場合は409
- 同時停止で二重更新しない
- 同時停止で監査ログを重複作成しない
- Transaction失敗時にRollbackする

動画編接続後の停止

- ready/published/未削除だけhiddenになる
- privateは変更しない
- hiddenは変更しない
- processingは変更しない
- failedは変更しない
- 削除済みは変更しない

- 変更したVideoごとに監査ログを作成する
- 途中失敗時にUser、Token、Video、AuditをRollbackする

利用再開

- suspended userを再開できる
 - Statusがactiveになる
 - TokenVersionを減らさない
 - Refresh Tokenを復活させない
 - hidden動画を公開しない
 - 再開監査ログを作成する
 - 既にactiveの場合は409
 - 同時再開で二重処理しない
 - Transaction失敗時にRollbackする
-

ブラウザ確認

管理者アカウント

1. 初期化処理を実行する。
2. adminが作成される。
3. 再実行する。
4. adminが増えない。
5. adminでログインする。
6. 管理者画面へ移動できる。

管理画面保護

1. 未認証で管理画面へ直接アクセスする。
2. Login画面へ遷移する。
3. 一般ユーザーで管理画面へアクセスする。
4. 管理画面を表示しない。

5. 一般ユーザーが管理者APIを直接呼ぶ。
6. 403になる。
7. adminでは表示できる。

ユーザー一覧・詳細

1. 一般ユーザーだけが一覧表示される。
2. adminは表示されない。
3. Name、Email、Status、CreatedAtが表示される。
4. 詳細画面へ移動できる。
5. 動画編前は投稿動画0件として表示される。
6. 動画編後は実際の投稿動画が表示される。

利用停止

1. activeユーザーを開く。
2. 停止理由を入力する。
3. 停止を実行する。
4. Statusがsuspendedになる。
5. 対象Userの既存Access Tokenが拒否される。
6. Refresh Token再発行が拒否される。
7. Loginが拒否される。
8. 監査ログが保存される。

利用再開

1. suspendedユーザーを開く。
2. 再開理由を入力する。
3. 再開を実行する。
4. Statusがactiveになる。
5. 停止前のTokenが利用できない。
6. 対象Userが改めてログインできる。

7. 再開監査ログが保存される。
8. hidden動画は自動公開されない。

セキュリティ確認

項目	対策
認証	Auth MiddlewareでJWT、User状態、TokenVersionを確認
認可	Admin Middlewareで現在のUser Roleを確認
Mass Assignment	RequestからRole、Status、TokenVersionを受け取らない
XSS	Reactの通常エスケープ、CSP、HTML直接挿入禁止
CSRF	Bearer認証の管理APIには不要。Cookie認証化時は追加
SQL Injection	GORMプレースホルダー、固定ORDER BY
Command Injection	初期化処理でShellコマンドを組み立てない
Body Limit	理由入力のJSONへ容量上限
排他制御	User行ロックで停止・再開を直列化
冪等性	同一状態への再操作は409、状態・監査ログを重複変更しない
Transaction	User、Token、Video、Auditを原子的に保存
監査ログ	操作者、対象、変更前後、理由、Request IDを保存
秘密情報	Password、Token、Cookie、Object KeyをResponse・Logへ出さない